

Interaktive Lernplattform für Grundlagen Informatik

Projektarbeit – SS 26

4. August 2026

Studenten:	Markus Nöth	markus.noeth@study.thws.de	5123027
	Richard Schröder	richard.schroeder@study.thws.de	5123042
	Enrico Ebert	enrico.ebert@study.thws.de	5123098
	Duy Tan Pham	duytan.pham@study.thws.de	5123063
	Kristian Popov	kristian.popov@study.thws.de	5123029

Dozent: Prof. Dr. Peter Braun

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Problemstellung (Author) . . . 1
1.2 Zielsetzung der Arbeit (Author) . . . 1
1.3 Methodik (Author) . . . 1

2 Übergreifende Architektur & Systemdesign **1**
2.1 Technologiestack (Author) . . . 1
2.2 Modulare Softwarearchitektur (Author) . . . 1
2.3 Unified UI-Konzept (Author) . . . 1
2.4 Simulator-Architektur (Author) . . . 1
2.5 Datenmodellierung (Author) . . . 1

3 Einsatz von Künstlicher Intelligenz im Projekt **1**
3.1 KI Reflexion (Markus Nöth) . . . 1
3.1.1 KI als Co-Developer (Markus Nöth) . . . 2
3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth) . . . 2
3.1.3 Kritische Reflexion (Markus Nöth) . . . 2
3.2 KI Reflexion (Richard Schröder) . . . 2
3.2.1 KI als Co-Developer (Richard Schröder) . . . 2
3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder) . . . 2
3.2.3 Kritische Reflexion (Richard Schröder) . . . 2
3.3 KI Reflexion (Enrico Ebert) . . . 2
3.3.1 KI als Co-Developer (Enrico Ebert) . . . 2
3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert) . . . 2
3.3.3 Kritische Reflexion (Enrico Ebert) . . . 2
3.4 KI Reflexion (Duy Tan Pham) . . . 2
3.4.1 KI als Co-Developer (Duy Tan Pham) . . . 2
3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham) . . . 3
3.4.3 Kritische Reflexion (Duy Tan Pham) . . . 3
3.5 KI Reflexion (Kristian Popov) . . . 3
3.5.1 KI als Co-Developer (Kristian Popov) . . . 3
3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov) . . . 4
3.5.3 Kritische Reflexion (Kristian Popov) . . . 4

4 Designentscheidungen der Fachmodule **5**
4.1 Turing Maschine (Author) . . . 5
4.1.1 Entscheidungen in der Architektur (Author) . . . 5
4.1.2 Entscheidungen im UI (Author) . . . 5
4.1.3 Andere Entscheidungen (Author) . . . 5
4.2 Endliche Automaten (Author) . . . 5
4.2.1 Entscheidungen in der Architektur (Author) . . . 5
4.2.2 Entscheidungen im UI (Author) . . . 5
4.2.3 Andere Entscheidungen (Author) . . . 5
4.3 Reguläre Sprachen (Author) . . . 5
4.3.1 Entscheidungen in der Architektur (Author) . . . 5
4.3.2 Entscheidungen im UI (Author) . . . 5
4.3.3 Andere Entscheidungen (Author) . . . 5
4.4 Zahlensysteme (Author) . . . 6
4.4.1 Entscheidungen in der Architektur (Author) . . . 6

4.4.2 Entscheidungen im UI (Author) . . . 6
4.4.3 Andere Entscheidungen (Author) . . . 6
4.5 Moore & Mealy Automaten (Author) . . . 6
4.5.1 Entscheidungen in der Architektur (Author) . . . 6
4.5.2 Entscheidungen im UI (Author) . . . 6
4.5.3 Andere Entscheidungen (Author) . . . 6
4.6 Disjunktive Normalform (Author) . . . 6
4.6.1 Entscheidungen in der Architektur (Author) . . . 6
4.6.2 Entscheidungen im UI (Author) . . . 6
4.6.3 Andere Entscheidungen (Author) . . . 6
4.7 Encoding (Author) . . . 6
4.7.1 Entscheidungen in der Architektur (Author) . . . 6
4.7.2 Entscheidungen im UI (Author) . . . 6
4.7.3 Andere Entscheidungen (Author) . . . 6
4.8 Java (Author) . . . 6
4.8.1 Entscheidungen in der Architektur (Author) . . . 6
4.8.2 Entscheidungen im UI (Author) . . . 6
4.8.3 Andere Entscheidungen (Author) . . . 6

5 Evaluation & Projekterfahrung 6
5.1 Qualitätssicherung (Author) . . . 6
5.2 Abgleich mit dem Lastenheft (Author) . . . 7
5.3 Lessons Learned (Author) . . . 7
5.4 Design-Iterationen (Author) . . . 7

6 Fazit & Ausblick 7
6.1 Zusammenfassung der Projektergebnisse (Author) . . . 7
6.2 Perspektiven (Author) . . . 7

Anhang i
Abkürzungsverzeichnis . . . i
Literatur . . . i

Abbildungsverzeichnis

Tabellenverzeichnis

1 Einleitung

Eine Arbeit von Technische Hochschule Würzburg-Schweinfurt (THWS) Studenten.

1.1 Problemstellung (Author)

Die Hürde der Abstraktion in Grundlagen Informatik

1.2 Zielsetzung der Arbeit (Author)

Didaktische Aufbereitung komplexer Modelle durch interaktive Simulationen

1.3 Methodik (Author)

Vorgehensweise im Team und Rollenverteilung

2 Übergreifende Architektur & Systemdesign

2.1 Technologiestack (Author)

React 19, TypeScript, Vite, Tailwind CSS 4, shadcn/ui, Zustand, i18next, Lucide Icons

2.2 Modulare Softwarearchitektur (Author)

Single Source of Truth (`moduleCatalog.tsx`), lazy Loading, Shared Topic Pattern – Sicherstellung der Erweiterbarkeit für neue Themen

2.3 Unified UI-Konzept (Author)

Wiederverwendbare Shared Components, konsistentes Designsystem (CSS-Variablen, Dark Mode, moduleigene Akzentfarben, Geist Font)

2.4 Simulator-Architektur (Author)

Das `ISimulator<TConfig, TState>`-Interface und der generische `useSimulationEngine`-Hook als Grundlage aller schrittweisen Simulationen

2.5 Datenmodellierung (Author)

Import/Export über `ISimulatorTransferCodec` mit Base64-Kodierung für Vorlesungskompatibilität

3 Einsatz von Künstlicher Intelligenz im Projekt

3.1 KI Reflexion (Markus Nöth)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.1.1 KI als Co-Developer (Markus Nöth)

Reflektieren und so...

3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth)

Reflektieren und so...

3.1.3 Kritische Reflexion (Markus Nöth)

Reflektieren und so...

3.2 KI Reflexion (Richard Schröder)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.2.1 KI als Co-Developer (Richard Schröder)

Reflektieren und so...

3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder)

Reflektieren und so...

3.2.3 Kritische Reflexion (Richard Schröder)

Reflektieren und so...

3.3 KI Reflexion (Enrico Ebert)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.3.1 KI als Co-Developer (Enrico Ebert)

Reflektieren und so...

3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert)

Reflektieren und so...

3.3.3 Kritische Reflexion (Enrico Ebert)

Reflektieren und so...

3.4 KI Reflexion (Duy Tan Pham)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.4.1 KI als Co-Developer (Duy Tan Pham)

Reflektieren und so...

3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham)

Reflektieren und so...

3.4.3 Kritische Reflexion (Duy Tan Pham)

Reflektieren und so...

3.5 KI Reflexion (Kristian Popov)

Mein Schwerpunkt lag während des Projekts auf den Modulen *Endliche Automaten* und *Reguläre Sprachen*. Ich habe dabei mehrere Modelle ausprobiert: Gemini 3.1 Pro, die kostenlose Version von ChatGPT sowie Claude Opus 4.8 und später Opus 5.

Jedes Modell hatte seine eigenen Stärken. Manche konnten theoretische Zusammenhänge besser erklären, andere waren stärker bei der Implementierung oder bei der Fehlersuche. Überwiegend habe ich mit Claude gearbeitet, weil das Modell bei Programmieraufgaben seltener fehlerhafte oder frei erfundene Vorschläge lieferte und ich die Antworten dadurch weniger oft nacharbeiten musste.

Erst mit der Zeit ist mir aufgefallen, dass die Modelle unterschiedlich viel Aufwand in die Erstellung einer Antwort investieren. Viele Modelle planen vor der eigentlichen Antwort mehr oder weniger ausführlich; der Planungsaufwand lässt sich meist in mehreren Stufen einstellen. Eine höhere Stufe bedeutet mehr verbrauchte Tokens, längere Wartezeit und höhere Kosten, führt aber nicht automatisch zu einem besseren Ergebnis. Für kleine, klar umrissene Änderungen war die niedrige Stufe ausreichend und zudem schneller. Erst wenn eine Aufgabe mehrere Dateien betraf oder eine Abwägung zwischen mehreren Lösungswegen verlangte, lieferte die höhere Stufe brauchbare Vorschläge. Deshalb habe ich je nach Aufgabe bewusst entschieden, welche Einstellung sinnvoll war.

3.5.1 KI als Co-Developer (Kristian Popov)

Die KI habe ich nicht über eine Chat-Oberfläche bedient, sondern direkt in die Entwicklungsumgebung eingebunden. Dadurch musste ich Code nicht mehr zwischen Editor und Chat kopieren. Stattdessen konnte das Modell auf die Projektdateien zugreifen, Änderungen vorschlagen und diese direkt im Projekt prüfen.

Besonders hilfreich war die KI bei wiederkehrenden Arbeiten: bei Refactorings und Optimierungen am Quellcode, beim Erzeugen von Boilerplate-Code, beim Schreiben von Unit-Tests sowie bei der Internationalisierung der Oberfläche. Gerade bei den Tests sparte sie viel Zeit. Für die Automatenlogik, das Parsen der Grammatiken und die Validierung der Programme konnte die KI aus der vorhandenen Implementierung schnell passende Testfälle ableiten, auch für Eingaben, an die ich selbst zunächst nicht gedacht hatte.

Außerdem half die KI beim Aufräumen des Projekts, etwa beim Beheben von TypeScript-Warnungen, beim Vereinheitlichen von Benennungen sowie beim Erstellen von Kommentaren und Methodendokumentationen. Mit den neueren Modellen und präzisen Prompts waren viele der vorgeschlagenen Änderungen bereits so gut, dass ich sie direkt übernehmen konnte. Trotzdem habe ich jede Änderung selbst überprüft und getestet.

Bei Architektur- und Implementierungsfragen habe ich die KI bewusst nicht selbst entscheiden lassen. Stattdessen ließ ich mir zunächst mehrere Lösungsansätze mit ihren Vor- und Nachteilen erklären und traf die Entscheidung anschließend selbst. Dieser Ansatz erforderte zwar etwas

mehr Zeit, führte aber zu besser begründeten Architekturentscheidungen und verhinderte, dass ich Vorschläge der KI ungeprüft übernahm.

Möglich wurde diese Arbeitsweise durch mehrere Erweiterungen aus dem offiziellen Plugin-Verzeichnis `claude-plugins-official`. Für meine Module waren vor allem zwei davon relevant:

`typescript-lsp`
: bindet den Sprachserver von TypeScript über das Language Server Protocol (LSP) ein. Der KI stehen dadurch dieselben Typinformationen und Referenzen zur Verfügung wie der Entwicklungsumgebung. Bezeichner und Signaturen musste das Modell also nicht aus dem Quelltext erschließen, sondern konnte gezielt danach suchen, wodurch Änderungen über Dateigrenzen hinweg konsistent blieben.

`chrome-devtools-mcp`
: verbindet die KI über das Model Context Protocol (MCP) mit den Entwicklerwerkzeugen des Browsers. Konsolenausgaben und das Verhalten der laufenden Anwendung ließen sich damit direkt auswerten. Bei der Fehlersuche in den Simulatoren war das der größte praktische Gewinn: Ein fehlerhafter Zustandsübergang wurde am tatsächlichen Ablauf sichtbar und musste nicht aus dem Quelltext rekonstruiert werden.

Zusätzlich habe ich das Projekt einmalig mit Graphify [1] analysiert und daraus einen Wissensgraphen erzeugt. Graphify verbindet eine statische Analyse auf Basis von Tree-sitter mit einer semantischen Auswertung durch ein Sprachmodell. Die Knoten des Graphen stehen für Dateien, Klassen und Funktionen, die Kanten für deren Beziehungen untereinander. Zu Beginn einer Sitzung musste die KI das Projekt nicht erneut vollständig einlesen, sondern konnte den für die jeweilige Aufgabe relevanten Teilgraphen abfragen. Gerade bei Änderungen, die mehrere Dateien betrafen, blieb der notwendige Kontext dadurch besser erhalten.

3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov)

Für beide Module musste jede Übung mehr enthalten als nur die eigentliche Aufgabenstellung. Zusätzlich wurden eine Referenzlösung, ein Startprogramm sowie Beispielwörter benötigt, mit denen die Lösung automatisch überprüft werden konnte. Erst diese Kombination ergibt ein Lernangebot, das ohne Betreuung funktioniert.

Die KI habe ich hier vor allem zur Variantenbildung eingesetzt. Ausgehend von einer selbst entworfenen Referenzaufgabe ließen sich weitere Aufgaben desselben Typs erzeugen, mit anderem Alphabet, anderer Wortlänge oder anderem Schwierigkeitsgrad. Genau dafür war die KI gut geeignet: Die Struktur steht fest, variiert wird nur der Inhalt.

Ob die Aufgaben tatsächlich korrekt waren, habe ich aber nicht der KI überlassen, sondern im Simulator geprüft. Diese Prüfung läuft deterministisch ab: Geprüft werden alle Wörter über dem gemeinsamen Alphabet bis zur Länge vier, und für jedes dieser Wörter muss der Automat der Studierenden dieselbe Entscheidung treffen wie die Referenzlösung. Bewertet wird damit das tatsächliche Verhalten des Automaten und nicht die Einschätzung eines Sprachmodells. Aus demselben Grund habe ich jede generierte Aufgabe an der Simulation gegengeprüft, bevor sie in den Aufgabenkatalog aufgenommen wurde.

3.5.3 Kritische Reflexion (Kristian Popov)

Die Vorschläge der KI waren oft brauchbar, aber nicht immer richtig oder passend für die bestehende Projektstruktur. Wiederholt schlug die KI APIs vor, die in der verwendeten Version so nicht mehr existierten. Gelegentlich erfand sie auch Funktionen oder Bibliotheken, die es in dieser Form nie gegeben hatte. Gerade Sonderfälle wurden häufig übersehen, und einzelne

Änderungen wirkten sich an Stellen aus, die im Vorschlag selbst gar nicht erwähnt waren. Besonders bei formalen Sprachen reicht es nicht aus, dass eine Lösung plausibel aussieht. Ob ein Automat die geforderte Sprache tatsächlich erkennt, zeigt sich erst beim Durchspielen konkreter Eingaben.

Ein weiteres Problem lag in der Struktur des Codes. Da sich *Endliche Automaten* und *Reguläre Sprachen* stark ähneln, duplizierte die KI bestehende Implementierungen, statt sie zu abstrahieren. So entstanden Simulator- und Übersichtsseiten, die sich zwischen den Modulen kaum unterschieden, und Funktionen, die nie in der Oberfläche eingebunden wurden und später wieder entfernt werden mussten. Betraf eine Änderung beide Module zugleich, etwa die einheitliche Gruppierung von Beispielen und Übungen nach Kategorien, reichte das Kontextfenster des Modells zudem nicht aus, um alle betroffenen Stellen zu erfassen. Solche Aufgaben musste ich in kleinere Schritte zerlegen.

Insgesamt hat mir die KI vor allem bei wiederkehrenden Aufgaben und beim Ausbau bereits vorhandener Funktionen viel Zeit gespart. Gerade bei theoretischen Konzepten und bei Änderungen an mehreren Modulen gleichzeitig war die erste Lösung aber selten schon die richtige. Den Wechsel zwischen den Modellen könnte künftig ein Orchestrator wie Sakana Fugu übernehmen, der je nach Aufgabe automatisch das passende Modell auswählt [2]. Das System erschien allerdings erst kurz vor Abschluss meiner Arbeit, sodass ich es nicht mehr einsetzen konnte. Für mich blieb die KI während des gesamten Projekts ein Werkzeug zur Unterstützung. Die fachlichen Entscheidungen und die Verantwortung für die Korrektheit der Implementierung lagen jedoch jederzeit bei mir.

4 Designentscheidungen der Fachmodule

4.1 Turing Maschine (Author)

4.1.1 Entscheidungen in der Architektur (Author)

4.1.2 Entscheidungen im UI (Author)

4.1.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.2 Endliche Automaten (Author)

4.2.1 Entscheidungen in der Architektur (Author)

4.2.2 Entscheidungen im UI (Author)

4.2.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.3 Reguläre Sprachen (Author)

4.3.1 Entscheidungen in der Architektur (Author)

4.3.2 Entscheidungen im UI (Author)

4.3.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.4 Zahlensysteme (Author)

4.4.1 Entscheidungen in der Architektur (Author)

4.4.2 Entscheidungen im UI (Author)

4.4.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.5 Moore & Mealy Automaten (Author)

4.5.1 Entscheidungen in der Architektur (Author)

4.5.2 Entscheidungen im UI (Author)

4.5.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.6 Disjunktive Normalform (Author)

4.6.1 Entscheidungen in der Architektur (Author)

4.6.2 Entscheidungen im UI (Author)

4.6.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.7 Encoding (Author)

4.7.1 Entscheidungen in der Architektur (Author)

4.7.2 Entscheidungen im UI (Author)

4.7.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.8 Java (Author)

4.8.1 Entscheidungen in der Architektur (Author)

4.8.2 Entscheidungen im UI (Author)

4.8.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

5 Evaluation & Projekterfahrung

5.1 Qualitätssicherung (Author)

CI/CD-Pipeline (GitLab Runner auf Raspberry Pi), automatisiertes Linting, Formatierung (Prettier), Unit-Tests (Vitest) und E2E-Tests (Playwright)

5.2 Abgleich mit dem Lastenheft (Author)

Wurden die Qualitätsziele (Performance, Usability, Barrierefreiheit durch Dark Mode und i18n) erreicht?

5.3 Lessons Learned (Author)

Herausforderungen in der Team-Kollaboration (Git-Workflow, Merge-Requests, geteilte Architektur) und technische Hürden

5.4 Design-Iterationen (Author)

Welche Entscheidungen mussten im Prozess revidiert werden?

6 Fazit & Ausblick

6.1 Zusammenfassung der Projektergebnisse (Author)

Sieben interaktive Fachmodule in zwei Lernbereichen

6.2 Perspektiven (Author)

Gamification (Badges/Punkte), LMS-Anbindung (Moodle/ILIAS) und Mobile Learning (responsive Optimierung)

Anhang

Abkürzungsverzeichnis

LSP Language Server Protocol; Protokoll zwischen Editor und Sprachserver für Codenavigation, Typinformationen und Refactoring

MCP Model Context Protocol; Offener Standard, über den Sprachmodelle externe Werkzeuge und Datenquellen anbinden

THWS Technische Hochschule Würzburg-Schweinfurt; Hochschule aus Würzburg und Schweinfurt (Bayern) [3]

Literatur

[1] S. Shamsi. „Graphify - Multi-modal Knowledge Graph Builder," besucht am 31. Juli 2026. Adresse: `https://graphify.net/`

[2] Sakana AI. „Sakana Fugu - Multi-agent System as A Model," besucht am 31. Juli 2026. Adresse: `https://sakana.ai/fugu/`

[3] Technische Hochschule Würzburg-Schweinfurt. „THWS - Technische Hochschule Würzburg-Schweinfurt," besucht am 14. Juni 2026. Adresse: `https://thws.de`